CheatCode

JAVA 8 :

1. Features of JAVA 8 and advantages?java8

Ans : - New Features added are :

- Lambda Expression
- Stream API
- Default Method and static methods in interface
- Functional Interface
- Optional class
- Method references

Advantages:
Compact code (Less boiler plate code)
More testable code
Parallel operations

What is Lambda Expression ?lambdacon
Ans: Lambda expression is an anonymous function i.e., (without name , return type and access modifier and having one lambda (->) symbol. It is used to provide the implementation of an interface which has functional interface. It saves a lot of code. In case of lambda expression, we don't need to define the method again for providing the implementation. Here, we just write the implementation code.
Eg. : Normal programming technique :

```
Public void add(int a , int b){
                System.out.println(a+b);
        }
```

Equivalent lambda be like :

```
BiConsumer<Integer, Integer> add = (a,b) -> System.out.println(a+b); //Lambda Expression and BiConsumer is a functional interface
add.accept(5, 8);
Predicate<Integer> noGreaterThan5 =  x -> x > 5;
```

NOTE: **Predefined Functional interface**

- **Biconsumer : - It is a predefined functional interface and it accepts only and returns nothing**.
- **Predicate is a functional interface, which accepts an argument and returns a boolean. Usually, it used to apply in a filter for a collection of objects.**

2. What is Functional Interface ?
Ans: Functional interfaces are those interfaces which can have only one abstract method and it can have any number of static method and default methods no restriction on that.
There are many Functional interfaces present in java such as :
Eg.: Comparable, Runnable, etc.

3. How Lambda expression and functional interface are related ?
Ans : So Functional interface is used to provide reference to lambda expression "->" this is the relation.

- **Comparator<String> c = (s1, s2) → s1.compareTo(s2);**
- (s1, s2) -> s1.compareTo(s2); ➔ This is lambda expression.
- Comparator<String> c ➔ This is Functional interface.

```
BiConsumer<Integer, Integer> add = (a,b) -> System.out.println(a+b);
add.accept(5, 8);
```

Hence for calling lambda expression we need Functional interfaces.

Functionalinterfacecon
4. How do we create our own functional interface and use lambda expression?
Ans : As we know functional interface is an interface with exactly one single abstract method and can have multiple static or default methods.

Example code:

```
@FunctionalInterface
public interface MyFunction {
    void myMethod(String input);
}
```

Lamda use:

```
MyFunction myFunction = (input) -> System.out.println("Input: " + input);
myFunction.myMethod("Hello, Lambda!");
```

5. What is method referencing ?

Ans : Method reference is used to refer method of functional interface. It is compact and easy form of lambda expression. Each time when you are using lambda expression to just referring a method, you can replace your lambda expression with method reference with "**::**" using double colon.
USE : Whenever we have existing implementation of abstract method of our functional interface then we go for method reference.

StreamCon

6. What is Stream API ?

Ans : **Java 8 Stream API Overview:**

- **Definition:** Java 8 Stream API is used to process collections of objects. It provides a sequence of objects with methods for pipelining operations to produce desired results.
- **Key Features:**
  - Doesn't change the original data structure.
  - Supports pipelined methods for streamlined operations.

Different Operations:

1. **Intermediate Operations:**
   - **Map:** Applies a given function to each element, returning a new stream.

```
List<Integer> numbers = Arrays.asList(2, 3, 4, 5);
List<Integer> squares = numbers.stream().map(x -> x * x).collect(Collectors.toList());
```

   - **Filter:** Filters elements based on a given predicate.

```
List<String> names = Arrays.asList("Reflection", "Collection", "Stream");
List<String> result = names.stream().filter(s -> s.startsWith("S")).collect(Collectors.toList());
```

   - **Sorted:** Sorts the stream elements.

```
List<String> names = Arrays.asList("Reflection", "Collection", "Stream");
List<String> result = names.stream().sorted().collect(Collectors.toList());
```

2. **Terminal Operations:**
   - **Collect:** Returns the result of intermediate operations.

```
List<Integer> numbers = Arrays.asList(2, 3, 4, 5, 3);
Set<Integer> squares = numbers.stream().map(x -> x * x).collect(Collectors.toSet());
```

   - **ForEach:** Iterates through every element of the stream.

```
List<Integer> numbers = Arrays.asList(2, 3, 4, 5);
numbers.stream().map(x -> x * x).forEach(y -> System.out.println(y));
```

   - **Reduce:** Reduces stream elements to a single value.

```
List<Integer> numbers = Arrays.asList(2, 3, 4, 5);
int evenSum = numbers.stream().filter(x -> x % 2 == 0).reduce(0, (ans, i) -> ans + i);
```

FlatMap:

- **Definition:** **flatMap()** is used to flatten or transform data within a stream. Unlike **map()**, it can handle nested structures.
- **Example:**

```
List<String> fruits = Arrays.asList("mango", "apple", "Guava", "Pomegranate");
fruits.stream().flatMap(str -> Stream.of(str.charAt(2))).forEach(System.out::println);
```

7. What is the difference b/w Map and FlatMap ? mapflatmapcon

Ans :

Criteria	map()	flatMap()
Processing	Processes a stream of values with one-to-one mapping	Processes a stream of values with one-to-many mapping and flattening
Mapper Function	Produces a single value for each input value	Produces multiple values for each input value
Data Transformation	From **Stream** to **Stream**	From **Stream<Stream>** to **Stream**
Usage Scenario	Used when the mapper function generates a single value	Used when the mapper function generates multiple values for each input value

- **Note: streamprogramcon**

1. Terminal Operation in Java 8:

- **Definition:** A terminal operation in Java 8 Stream API produces a result or a side-effect, and it follows the execution of intermediate operations.
- **Key Points:**
  - Examples: **forEach**, **reduce**, **collect**, **sum**, **average**.
  - Triggered after a series of intermediate operations.
  - Produces a result or side-effect.

2. Memory Allocation in Java 8:

- **Overview:** In Java 8, memory allocation is managed by the Garbage Collector, a part of the Java Virtual Machine (JVM).
- **Key Points:**
  - Automatic memory management.
  - Garbage Collector identifies and removes unreferenced objects.
  - Efficient memory usage, reducing manual memory management issues.

3. HashMap Features in Java 8:

- **Java 8 HashMap Enhancements:**
  1. **Collision Handling:** Improved handling of collisions using a balanced tree structure for better performance.
  2. **Performance:** Improved performance for scenarios with a high number of key collisions.
  3. **Stream API Integration:** Enhanced integration with Java 8 Stream API for concise operations.

Defaultinterfacecon

8. Default Method in Interface:

- **Definition:** Default methods in interfaces were introduced in Java 8 to allow interfaces to have methods with implementation without affecting the classes that implement the interface.
- **Syntax:**

```
default void methodName() {
   // Statement
}
```

- **Advantages:**

- Provides backward compatibility for old interfaces.
- Allows interfaces to have methods with default implementations.
- Interfaces can also have static methods.

9. Diamond Problem:

- **Definition:** The Diamond Problem is a multiple inheritance problem where ambiguity arises when a child class object cannot call two parent class methods with the same signature.
- **Solution:** Interfaces with default methods help solve the Diamond Problem. If interfaces contain default methods with the same signature, implementing classes must explicitly specify which default method to use or override the default method.

10. Static Method in Interface:

- **Definition:** Static methods in interfaces are defined with the keyword **static**. They cannot be overridden in implementation classes.
- **Syntax:**

InterfaceName.staticMethodName;

- **Uses:**
  - Provides security by not allowing implementing classes to override static methods.
  - Useful for utility methods, such as null checking, where the definition will not change.

11. Optional Class in Java 8:

- **Definition:** The **Optional** class is a container object used to contain not-null objects, addressing the issue of **NullPointerException**. It is a public final class in the **java.util** package.
- **Key Methods:**
  - **isPresent()**: Returns true if the **Optional** instance contains a value.
  - **get()**: Returns the value contained in the **Optional** instance, throwing an exception if not present.
  - **orElse()**: Returns the value if present, otherwise returns a default value.
  - **orElseGet()**: Returns the value if present, otherwise invokes a supplier function.
  - **ifPresent()**: Performs an action on the value if it exists.

❖ **Program for Employee salary = 50000**

```
List<String> empstrm = emplist.stream().filter(employee -> employee.salary==50000).map(employee -> employee.name).collect(Collectors.toList());
empstrm.forEach(x->System.out.println(x));
```

❖ ***Write a program to print 5th max salary of the Employee?*** asked in the interview....**

```
Optional<Engineers> eng = emplist.stream().sorted(Comparator.comparingDouble(Engineers :: getSalary).reversed()).skip(1).findFirst();
System.out.println("The 5th maximum salary : "+ eng);
```

❖ **Program to print Max/Min employee salary from given Collection ?**

```
Optional<Engineers> engmax = emplist.stream().max(Comparator.comparing(Engineers :: getSalary));
        System.out.println("Maximum salary : "+engmax);
```

```
Optional<Engineers> engmin = emplist.stream().min(Comparator.comparing(Engineers:: getSalary));
System.out.println("Minimum Salary : "+engmin);
```

❖ **Program to print employees count working in each department?**

```
Map<String, Long> empstrm2 =
emplist.stream().collect(Collectors.groupingBy(Engineers::getDept, Collectors.counting()));
empstrm2.entrySet().forEach(entry -> System.out.println(entry.getKey()+""+entry.getValue()));
```

- **Group a list of users by their country and then calculate the average age of users in each country?**

```
List<User> users = Arrays.asList(
    new User("Alice", "USA", 25),
    new User("Bob", "Canada", 30),
    new User("Carol", "USA", 40),
    new User("Dave", "Canada", 50),
    new User("Eve", "USA", 60)
);

// Group the users by country.
Map<String, List<User>> usersByCountry =
users.stream().collect(Collectors.groupingBy(User::getCountry));

// Calculate the average age of users in each country.
Map<String, Double> averageAgeByCountry = usersByCountry.entrySet().stream()
    .collect(Collectors.toMap(entry -> entry.getKey(), entry ->
entry.getValue().stream().mapToDouble(User::getAge).average().orElse(0.0)));

// Print the average age of users in each country.
for (Map.Entry<String, Double> entry : averageAgeByCountry.entrySet()) {
    System.out.println(entry.getKey() + ": " + entry.getValue());
}
```

```
USA: 41.666666666666664
Canada: 40.0
```

- **Find the top 10 most popular products, based on the number of times they have been purchased?**

```
List<Product> products = Arrays.asList(
    new Product(1, "Product A", 100),
    new Product(2, "Product B", 200),
    new Product(3, "Product C", 300),
    new Product(4, "Product D", 400),
    new Product(5, "Product E", 500)
);

// Find the number of times each product has been purchased.
Map<Integer, Long> purchaseCountByProduct = products.stream()
    .collect(Collectors.groupingBy(Product::getId, Collectors.counting()));

// Sort the products by purchase count in descending order.
List<Product> sortedProducts = purchaseCountByProduct.entrySet().stream()
    .sorted(Comparator.comparing(Map.Entry::getValue, Comparator.reverseOrder()))
    .map(entry -> products.stream().filter(product -> product.getId() ==
entry.getKey()).findFirst().get())
    .collect(Collectors.toList());

// Print the top 10 most popular products.
for (int i = 0; i < 10; i++) {
    Product product = sortedProducts.get(i);
    System.out.println(product.getId() + ": " + product.getName());
}
```

```
5: Product E
```

```
4: Product D
3: Product C
2: Product B
1: Product A
```

- **Find the longest word in a list of strings:**

```
List<String> words = Arrays.asList("hello", "world", "this", "is", "a", "list", "of", "strings");

// Find the longest word in the list.
String longestWord =
words.stream().max(Comparator.comparingInt(String::length)).orElse("");

// Print the longest word.
System.out.println(longestWord);
```

```
strings
```

- **Find the most common word in a list of strings, ignoring case?**

```
List<String> words = Arrays.asList("hello", "world", "this", "is", "a", "list", "of", "strings", "hello",
"world");

// Find the most common word in the list, ignoring case.
String mostCommonWord =
words.stream().collect(Collectors.groupingBy(String::toLowerCase, Collectors.counting()))
   .entrySet().stream()
   .max(Comparator.comparing(Map.Entry::getValue))
   .map(Map.Entry::getKey)
   .orElse("");

// Print the most common word.
System.out.println(mostCommonWord);
```

```
hello
```

Static Concepts:
Staticcon

12. Static in Java:
- Static: Non-access modifier for memory management.
- Applied to variables, methods, nested classes, and initialization blocks (static block).

13. Static Variable:
- Allocated memory once during class loading.
- Shared by all instances of the class.
- Accessed directly without creating an instance.

14. Static Method:
- Belongs to the class rather than an object.
- Called using the class name.
- Accesses static variables directly.
- Cannot access non-static variables or methods.

15. Why `main()` Method is Declared as Static:
- JVM calls `main()` automatically.
- No need to create an object, avoids extra memory allocation.

16. Command Line Arguments:
- Arguments passed to a program at runtime.
- Accessed using `main(String[] args)`.

Constructors and Methods:

17. Constructor:
- Special method for creating and initializing objects.
- Types:
  - No-Arg Constructor: Default, without arguments.
  - Parameterized Constructor: With one or more parameters.

18. Difference Between Constructors and Methods:

Methods	Constructors
Exhibits functionality.	Initializes objects.
Invoked explicitly.	Invoked implicitly.
May or may not return a value.	No return value.
No default method provided if absent.	Default constructor provided if absent.
Name should not be the same as the class.	Name must be the same as the class.

JVM Architecture:

JVM Working:
- Java applications follow WORA (Write Once Run Anywhere).
- JVM is a part of JRE (Java Runtime Environment).
- JVM runs Java bytecodes and calls the `main` method.

JDK vs. JRE:
- JDK (Java Development Kit):
  - Software development environment.
  - Includes JRE, compiler (`javac`), interpreter/loader (`java`), archiver (`jar`), documentation generator (`Javadoc`), and more.

- JRE (Java Runtime Environment):
  - Minimum requirements for executing Java applications.
  - Consists of JVM, core classes, and supporting files.

JVM Architecture:
- Java Source Code (.java): Compiled by `javac`.
- .class file (Java Bytecode): Executed by the JVM.
- Steps: Compilation → Bytecode Generation → Class Loader → Bytecode Verifier → Interpreter → JIT Compiler → Runtime.

MEMORY allocations:
- The code section contains your bytecode.
- The Stack section of memory contains methods, local variables, and reference variables.
- The Heap section contains Objects (may also contain reference variables).
- The Static section contains Static data/methods.
- The String constant Pool contains String literals and new object of string and string is also contains in stack.

Note:
- When a method is called, a frame is created on the top of the stack.
- Once a method has completed execution, the flow of control returns to the calling method and its corresponding stack frame is flushed.
- Local variables are created in the stack.
- Instance variables are created in the heap & are part of the object they belong to.
- Reference variables are created in the stack.

OOPs Concept(Core JAVA):
oopscon

8. What is Java? What is class and object in java?

Ans : Java is object oriented programming language which supports the following fundamental concepts like

- ❖ Polymorphism
- ❖ Inheritance
- ❖ Abstraction
- ❖ Encapsulation

Class : - A class is defined as blueprint which describes thee behavior or state that the object of its type supports.

Object :- An object is nothing but a real world entity which have both state and behavior.

Inheritance : inheritancecon

Inheritance

- **Definition:** Inheritance is the mechanism by which a class (subclass or derived class) acquires the properties and behaviors of another class (superclass or base class).
- **Uses:**
  1. Achieving method overriding for runtime polymorphism.
  2. Enhancing code reusability.
- **Types:**
  - Single Inheritance
  - Multilevel Inheritance
  - Hierarchical Inheritance
  - Hybrid Inheritance
  - Multiple Inheritance (not supported in Java)

Polymorphism

- **Definition:** Polymorphism refers to the ability of objects to take on multiple forms. It allows one task to be performed in different ways.
- **Achievement:** Through method overloading and method overriding.

Static Binding vs. Dynamic Binding

Static Binding	Dynamic Binding
Resolved at compile time	Resolved at runtime
Uses type of the class and fields	Uses object to resolve binding
Example: Overloading	Example: Method overriding
Applies to private, final, and static methods/variables	Applies to virtual methods

Method Overloading vs. Method Overriding

Method Overloading	Method Overriding
Multiple methods with the same name but different parameters	Same method signature in superclass and subclass
Compile-time polymorphism	Runtime polymorphism
Enhances method behavior	Changes existing method behavior
No inheritance required	Requires inheritance (IS-A

Method Overloading	Method Overriding
	relationship)

Overriding for Access Modifiers

- Private < Default < Protected < Public

Method Hiding

- Occurs with static methods in subclasses.
- Subclass static method hides superclass static method.

Encapsulation

- **Definition:** Encapsulation involves wrapping code and data into a single unit. It facilitates data hiding and provides getter and setter methods for controlled access.
- **Advantages:**
  - Flexibility and ease of change
  - Unit testing
  - Controlled access and security
  - Immutable class creation

Abstract vs. Interface

Abstract Class	Interface
Partial abstraction	Full abstraction
Supports single inheritance	Supports multiple inheritance
Extends with **extends** keyword	Implements with **implements** keyword
Can have various access modifiers and static/final variables	By default, public methods and static/final variables

Note: Access modifier accessmodifiercon

Modifiers/Access	class	package	subclass	global
public	**Yes**	**Yes**	**Yes**	**Yes**
protected	**Yes**	**Yes**	**Yes**	**No**
default	**Yes**	**Yes**	**No**	**No**
private	**Yes**	**No**	**No**	**No**

This and Super thissupercon

9. Difference b/w this() and super() ?

Ans :

This()	Super()
➢ this() represent current class instance of sub class and used to point the current class instance ➢ this() is used to call default constructor of same class ➢ this() is used to access methods of current class	➢ super() represents the current instances of parent class and used to point super class instance ➢ super() is used to call default constructor of the parent class ➢ super() is used to access method of the base class.

Association

- **Definition:** Association is a relationship between two separate classes established through their objects. It can be one-to-one, one-to-many, many-to-one, or many-to-many.

Aggregation

- **Definition:** Aggregation is a special form of association where one class has a "has-a" relationship with another class. It is a unidirectional association, and both entities can survive independently.

Composition

- **Definition:** Composition is a restricted form of aggregation where one class contains another class as a part. It represents a "part-of" relationship, and both entities are highly dependent on each other.

Aggregation vs. Composition

Aggregation	Composition
Represents a weak relationship	Represents a strong relationship
Unidirectional association	Bidirectional association
Both entities can survive independently	Composed object cannot exist without the other entity
Loosely coupled	Tightly coupled

Object Class

- **Definition:** The Object class is present in the **java.lang** package and acts as the root of the inheritance hierarchy in Java. Every class in Java is directly or indirectly derived from the Object class.
- **Methods:**
  - **equals()**: Compares two objects for equality.
  - **hashCode()**: Returns a hash code value for the object.
  - **toString()**: Returns a string representation of the object.
  - **clone()**: Creates a shallow copy of the object.
  - **wait()**: Causes the current thread to wait until it is notified.
  - **notify()**: Wakes up a single thread that is waiting on this object's monitor.
  - **notifyAll()**: Wakes up all threads that are waiting on this object's monitor.

10. Difference between Static and Final keywords?

• Feature	static	final
Applied To	Methods, Variables, Nested Classes	Variables, Methods, Classes
Usage	Memory management, Class-level members	Constants, Inheritance (for classes/methods)
Memory Allocation	Allocated memory once (class-level)	Memory allocated once (variables/constants)
Access	Accessed using class name or instance	Accessed using the name directly (no instance needed)
Modifiability	Can be modified (except for final methods)	Cannot be modified once assigned
Inheritance	Inherited by subclasses	Inherited by subclasses (for methods, prevents override)
Example	**static int count;**	**final int MAX_VALUE = 100;**

Both **static** and **final** serve different purposes: **static** for memory management and shared members, and **final** for constants and immutability.

Marker Interface markerinterfacecon
Marker Interface or Tagged Interface

Marker Interface:
- An interface with no methods, fields, or constants is known as a marker interface or tagged interface.
- It serves as a tag for the classes that implement it.
- Marker interfaces are used to provide metadata to the classes.

Examples:
- `Serializable` and `Cloneable` are examples of marker interfaces in Java.

Cloneable Interface

Cloneable Interface:
- `Cloneable` is a marker interface in Java.
- It indicates that the instances of the class implementing it can be cloned, i.e., an exact copy of the object can be created.

Example:
```
class CloneableExample implements Cloneable {
    // Class implementation
    @Override
    protected Object clone() throws CloneNotSupportedException {
        return super.clone();
    }
}
```

Serializable Interface

Serializable Interface:
- `Serializable` is another marker interface in Java.
- It indicates that the instances of the class implementing it can be serialized, i.e., converted into a byte stream.

Example:
```
import java.io.Serializable;

class SerializableExample implements Serializable {
    // Class implementation
}
```

Shallow Copy vs. Deep Copy

Shallow Copy:
- A shallow copy creates a new object, but does not create copies of nested objects.
- Changes made to the cloned object will be reflected in the original object and vice versa.

Deep Copy:
- A deep copy creates a new object and also creates copies of all nested objects.
- Changes made to the cloned object will not affect the original object, and vice versa.

Additional Notes

- Serialization involves converting the state of an object into a byte stream.
- Deserialization is the reverse process, converting a byte stream back into an object.
- `SerialVersionUID` is a unique identifier used during deserialization to ensure compatibility.

- Implementing `Serializable` in a class allows instances of that class to be serialized.

Summary

- Marker interfaces are used to provide metadata to classes.
- `Cloneable` indicates that objects can be cloned.
- `Serializable` indicates that objects can be serialized.
- Shallow copy reflects changes between original and clone; deep copy does not.
- Serialization involves converting objects to byte streams and back.

Strings : -
Stringcon
String and Immutable Class Concepts Summary

String:
1. String Overview:
- A string is an array of characters and is an immutable class.
- It follows the principle of flyweight, and JVM creates a string object in the String pool.

2. String vs. StringBuffer vs. StringBuilder:
- **String:**
  - Immutable.
  - Slow and consumes more memory when concatenating many strings.
  - Overrides the `equals()` method of the `Object` class.
- **StringBuffer:**
  - Mutable.
  - Synchronized and thread-safe.
  - Less efficient than `StringBuilder`.
- **StringBuilder:**
  - Mutable.
  - Not synchronized and not thread-safe.
  - More efficient than `StringBuffer`.

3. Why String is Immutable:
- Due to the concept of string literal, where changes to the object affect all reference variables.

Immutable Class:
1. Immutable Class Definition:
- An immutable class is one whose objects, once created, cannot be modified.
- Examples include wrapper classes (Integer, Boolean, etc.) and the `String` class.

2. Creating Immutable Class:
- Declare the class as `final` to prevent extension.
- Make fields `private` and `final` to disallow direct access and modification.
- Initialize fields in the constructor.
- Provide only getter methods for fields; avoid setter methods.

3. Immutable Class Example:

```
public final class ImmutableClass {
    private final int value;

    public ImmutableClass(int value) {
        this.value = value;
    }
    public int getValue() {
        return value;
    }
}
```

Design Patterns: designpatterncon

11. What is Singleton design pattern and how do we create it?singletoncon

Ans: The Singleton design pattern ensures only one instance of a class is created and provides a global point of access to it. To implement:

Example code:

```
public final class Singleton {

    private static Singleton instance;

    private Singleton() {}  // Private constructor

    public static Singleton getInstance() {

        if (instance == null)

            instance = new Singleton();

        return instance;

    }

}
```

1. Declare the class as final.
2. Make the constructor private.
3. Create a static variable to store the single instance.
4. Provide a static method to get or create the instance.

Benefits:
- Improves performance by reducing object creation.
- Enhances code readability and maintainability.
- Prevents errors by ensuring a single instance.

Drawbacks:
- May make testing and debugging challenging.
- Can lead to tight coupling between code parts.
- May limit scalability.

Example use case: Managing a database connection where only one connection instance is needed.
Remember: Declare as final, private constructor, static variable, and a static method for access.

48. What is Factory design pattern and how do we create with benefits and drawbacks?

Ans: Factory Design Pattern:

1. **Purpose**:
   - Creational pattern for creating objects without specifying their exact classes.

2**. Implementation Steps**:

1. Create a base interface or abstract class for target objects.
2. Implement concrete classes representing different object types.
3. Develop a factory class with methods for creating each object type.
4. Use the factory class to create desired objects.

3. **Example Code:**

```
public interface Shape {
    void draw();
}

public class Circle implements Shape {
    @Override
    public void draw() {
        System.out.println("Drawing a circle");
    }
}

public class Square implements Shape {
    @Override
    public void draw() {
        System.out.println("Drawing a square");
    }
}

public class ShapeFactory {
    public Shape createShape(String shapeType) {
        if (shapeType.equals("circle")) {
            return new Circle();
        } else if (shapeType.equals("square")) {
            return new Square();
        } else {
            throw new IllegalArgumentException("Unsupported shape type: " + shapeType);
        }
    }
}
```

4. **Usage**:

```
ShapeFactory factory = new ShapeFactory();

Shape circle = factory.createShape("circle");
Shape square = factory.createShape("square");

circle.draw(); // Drawing a circle
```

```
square.draw(); // Drawing a square
```
Remember: Factory Design Pattern allows flexible object creation by centralizing instantiation logic, promoting code scalability and maintainability.

12. Describe SOLID design principle ?

Ans: SOLID Principles:

1**. Single Responsibility Principle (SRP):**
- One class, one responsibility.
- e.g., Separate database connection management from user input validation.

2. **Open/Closed Principle (OCP):**
- Open for extension, closed for modification.
- e.g., Use interfaces and polymorphism for adding new shapes to a drawing class.

3. **Liskov Substitution Principle (LSP):**
- Subtypes should replace their base types without breaking code.
- e.g., Subclass of Shape should seamlessly substitute a Shape object.

4. **Interface Segregation Principle (ISP):**
- Clients should not depend on interfaces they don't use.
- e.g., Break down a large user management interface into smaller, specific interfaces.

5. **Dependency Inversion Principle (DIP):**
- Depend on abstractions, not concretions.
- e.g., Use an EmailSender interface for sending emails to make the code flexible and reusable.

Application Examples:

1. SRP Example:
- A class managing database connections should not handle user input validation.

2. OCP Example:
- Allow extending a shape drawing class without modifying existing code using interfaces.

3. LSP Example:
- Subclass of Shape should implement all methods of the Shape class without breaking expectations.

4. ISP Example:
- Break down a large user management interface into smaller ones like authentication and permissions.

5. DIP Example:
- A class sending emails should depend on an EmailSender interface, enabling easy switch to different email services.

Remember: Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion principles lead to modular, reusable, maintainable, and testable code.

Exception exceptioncon

13. Exception Overview:

- **Definition:** An exception is an abnormal condition during program execution, disrupting the normal flow. It must be handled to prevent abrupt termination.
- **Key Points:**
  - Occurs during program execution.

- Disrupts normal flow.
- Requires proper handling to avoid abrupt termination.

- **Hierarchy:**
  - Object class is the parent class of the Throwable class.

35. Checked vs. Unchecked Exceptions:

Checked Exceptions	Unchecked Exceptions
Known to compiler, checked at compile time	Not known to compiler, occurs at runtime
Examples: IOException, SQLException	Examples: ArithmeticException, NullPointerException

36. Difference between Error and Exception:

Errors	Exceptions
Caused by the environment	Caused by the application itself
Example: OutOfMemoryError	Example: NullPointerException

37. Exception Handling Mechanism:

- **Try Block:** Contains code monitored for exceptions.
- **Catch Block:** Catches exceptions from the try block.
- **Finally Block:** Always executed, whether an exception occurs or not.

Note:

1. **Multiple Catch Blocks:** Handle different types of exceptions.
2. **Nested Try Statement:** A try-catch-finally block inside another.

38. throw vs. throws:

throw	**throws**
Explicitly throws an exception instance	Declares exceptions in a method signature
Used within the method	Used with method signature
Followed by an instance	Followed by class
Used for custom exceptions	Declares exceptions for method propagation
Example: **`throw new IOException("Message");`**	Example: **`void method() throws IOException {}`**

Note:

- **Exception Propagation:** Uncaught exceptions are processed by the default handler.

- **Custom Exception:** Created using **throw new ExceptionType("Message");**

Throwable Exception:

- **Short Answer:** Using **Throwable** as a generic exception is not recommended. It includes both **Error** and **Exception**, leading to potential misuse.
- **Notes:**
  1. **Throwable Hierarchy:**
     - **Throwable** is the root class for all errors and exceptions.
     - It has two main subclasses: **Error** and **Exception**.
  2. **Error vs. Exception:**
     - **Error**: Represents critical errors in the system, often irrecoverable (e.g., **OutOfMemoryError**).
     - **Exception**: Represents exceptional conditions that can be handled (e.g., **IOException**).
  3. **Misuse of Throwable:**
     - Using **Throwable** as a catch type catches both errors and exceptions.
     - May lead to inappropriate handling of critical errors.
  4. **Prefer Specific Exceptions:**
     - It's recommended to catch specific exceptions relevant to the situation.
     - Enables precise error handling and better code readability.

Summary:

- **Throwable** is the root class for errors and exceptions.
- Includes **Error** and **Exception** subclasses.
- Misusing **Throwable** may lead to inappropriate handling.
- Prefer catching specific exceptions for better code control.

14. Difference b/w final, finally and finalize ?
Ans:

Final	Finally	Finalize
➢ It is used to apply restriction on class so it cannot extended, method so it cannot be overridden and variable so values cannot be changed. ➢ It is a keyword	➢ It is used to place important code it will be executed whether exception is handled or not. ➢ It is a Block	➢ It is used to perform cleanup processing just before object is garbage collected. ➢ It is a method.

Note: i) constructors cannot be final
ii) A final class is very useful when you want a high level of security in your application
iii) we can change the state of an object to which a final reference variable is pointing, but we can't re-assign a new object to this final reference variable.
iv) we can't change the value of an interface field.

15. What Are The Rules Of Exception Handling With Respect To Method Overriding?
Exceptionrulescon
Ans: ### Exception Handling Rules in Method Overriding:

1. Superclass No Exception:
- If the superclass method doesn't declare any exception (`throws`), the subclass method cannot declare checked exceptions but can declare unchecked exceptions.

2. Superclass Declares Exception:
- If the superclass method declares an exception using `throws`:

a) Subclass can declare the same exception as the superclass method.
b) Subclass can declare a subclass exception of the one declared in the superclass method.
c) Subclass cannot declare any exception if the superclass method declares a checked exception, but it's allowed for unchecked exceptions.

Remember:
- No checked exceptions in the subclass if the superclass doesn't declare any.
- Options for checked exceptions in the subclass when the superclass declares them: same, subclass, or none.
- Unchecked exceptions can always be declared in the subclass.

Multithreading:

Java Multithreading and Synchronization Quick Notes:

56. Multithreading and Threads in Java:
- **Multithreading:** Executing multiple threads simultaneously.
- **Threads:**
  - Lightweight processes.
  - Enable multiprocessing.
  - Each program has at least one thread (main thread).
  - Threads run in parallel.
  - Have their own stack.

16. Process vs. Threads:

Process	Threads
Heavyweight, independent execution unit.	Lightweight, shares resources within a process.
Takes more time to terminate.	Takes less time to terminate.
Isolated memory space.	Shares memory with other threads.

58. Thread Life Cycle and States:
- Thread States:
  - **New**: Created but not started.
  - **Runnable**: Eligible to run.
  - **Running**: Currently executing.
  - **Waiting/Non-Runnable**: Waiting for another thread, sleeping, or blocked.
  - **Terminated** (Dead):Execution completed.

59. Implementing Threads in Java:
- Two ways:
  1. Implement `Runnable` interface.
  2. Extend `Thread` class.

60. Extending Thread Class vs. Implementing Runnable Interface:
- **Extending Thread**:
  - Cannot extend other classes.
  - Each thread has a unique object.
- **Implementing Runnable**:
  - Allows extending other classes.
  - Shares the same object among threads.

61. Thread Behavior Unpredictability:
- Thread behavior is unpredictable due to the thread scheduler, which may vary across platforms.

62. ThreadPool:
- A pool of threads that reuses a fixed number of threads for executing tasks.
- Improves performance by reusing threads instead of creating new ones.

63. Starting Thread Twice:
- Cannot start a thread again; results in `IllegalThreadStateException`.

64. Calling `run()` Directly:
- Calling `run()` directly runs the method in the current thread, not in a separate thread.

65. Overriding `start` Method:
- If `start` method is overridden, `run()` won't be called until explicitly invoked.

66. Difference between `Thread.run()` and `thread.start()`:
- `**thread.start**()`: Creates a new thread and executes `run()` on the new thread.
- `**thread.run**()`: Executes `run()` on the current thread without creating a new one.

67. Deadlock Situation and Avoidance:
- Deadlock: Threads waiting for each other to finish executing.
- Avoidance:
  - Avoid nested locks.
  - Avoid unnecessary locks.
  - Use thread join.

68. `yield()`, `sleep()`, and `join()` Methods:
- `yield()`: Allows other threads to run if the current thread isn't doing anything important.
- `sleep()`: Pauses the current thread for a specified time.
- `join()`: Waits for a thread to die.

69. Thread Priority and Daemon Thread:
- Thread Priority: Range 1 to 10; higher priority executes first.
- Daemon Thread: Provides services to user threads, terminates when no user threads exist.

70. Why JVM Terminates Daemon Thread:
- Daemon threads exist to serve user threads. If no user threads, no need for daemon threads.

71. Invoking Garbage Collection in Java:
- Can be invoked manually:
  - `System.gc()`
  - `Runtime.getRuntime().gc()`

72. Synchronization in Java:
- Controls access of multiple threads to shared resources.
- Keyword: `synchronized`.
- Critical section: Only one thread executes at a time.

73. Inter-Thread Communication in Java:
- Implemented by `wait()`, `notify()`, and `notifyAll()` methods.
- Pauses a thread and allows another to execute.

74. Differences between `wait()` and `sleep()`:
- **wait**():
  - Non-static method.
  - Releases lock.
  - Requires synchronized context.
-**sleep**():
  - Static method.
  - Does not release lock.
  - Can be used without synchronization.

Collections

42. ArrayList vs. LinkedList:

ArrayList	LinkedList
Slower in manipulation due to bit shifting	Faster in manipulation, no bit shifting
Faster in storing and accessing data	Slower in accessing data, efficient in manipulation
Implements List interface	Internally node-based, implements List and Queue interface

Additional Notes:

- **Queue:** Ordered list for inserting elements at the end and deleting from the start (FIFO).
- **ArrayList:** Initial capacity of 10, increases by 50% when filled.
- **LinkedList (SLL):** Node-based, moving in forward direction.

43. Comparable vs. Comparator:

Comparable	Comparator
Single sorting sequence	Multiple sorting sequences
Affects the original class	Requires another class, no modification to the original class
Uses **compareTo()** for sorting	Uses **compare()** for sorting
In **java.lang** package	In **java.util** package

Examples:

```
// Comparable
public class NameComparator implements Comparator<Country> {
    @Override
    public int compare(Country c1, Country c2) {
        return c1.name.compareTo(c2.name);
    }
```

```
// Comparator
public class PopulationComparator implements Comparator<Country> {
    @Override
    public int compare(Country c1, Country c2) {
        return Integer.compare(c1.population, c2.population);
    }
```

44. List vs. Set vs. Map:

List	Set	Map
Allows duplicate elements	Doesn't allow duplicate elements	Key-Value pairs
Maintains insertion order	No specific order maintained	No specific order, keys or values
Examples: ArrayList, LinkedList	Examples: HashSet, TreeSet	Examples: HashMap, TreeMap

45. HashSet vs. HashMap:

HashMap	HashSet
Key-Value pairs	Only stores keys
No duplicate keys allowed	No duplicate keys allowed
Example: HashMap	Example: HashSet

46. Hashtable vs. HashMap:

Hashtable	HashMap
Synchronized, thread-safe	Not synchronized, not thread-safe
No null keys or values	Allows one null key, multiple null values
Example: Hashtable	Example: HashMap

47. HashMap Internal Working:

- HashMap uses an array of nodes, each containing Hashcode, Key, Value, and reference to other nodes or null.
- Hashing is used to calculate the index for storing Key-Value pairs.
- Hash collision is resolved by forming a LinkedList at the same index.
- **put** appends objects to the LinkedList, and **get** checks equality using **equals()**.

48. ConcurrentModificationException:

- Occurs when an object is modified concurrently.
- Common in Java Collection classes during iteration.
- Avoided using fail-safe iterators (e.g., **ConcurrentHashMap**, **CopyOnWriteArrayList**).

49. Fail-safe vs. Fail-fast Iterator:

Fail-safe	Fail-fast

Fail-safe	Fail-fast
Throws exception on modification	Allows modification, no exception
Uses a copy of the collection	Uses the original collection
Examples: ConcurrentHashMap, CopyOnWriteArrayList	Examples: ArrayList, HashMap

50. Runnable vs. Callable:

Runnable	Callable
Override **run()** method	Override **call()** method
Cannot return a result	Can return a result
Belongs to **java.lang** package	Belongs to **java.util.concurrent** package

Examples:

```
// Runnable
class MyRunnable implements Runnable {
    public void run() {
        // Task logic
    }
}
```

```
// Callable
class MyCallable implements Callable<Integer> {
    public Integer call() {
        // Task logic, return result
    }
}
```

Important Data Structures:

1. **Queue:** Follows FIFO principle, used for buffers.
2. **Deque:** Double-ended queue, supports insertion and removal at both ends.
3. **Stack:** Follows LIFO principle, used for function calls and backtracking.
4. **HashSet:** Stores unique elements using a hash table.
5. **LinkedHashSet:** Maintains insertion order, implemented using a linked list.
6. **TreeSet:** Maintains sorted order, implemented using a balanced binary search tree.
7. **HashMap:** Stores key-value pairs using a hash table.
8. **LinkedHashMap:** Maintains insertion order, implemented using a linked list.
9. **TreeMap:** Maintains sorted order for key-value pairs.

NOTE :

- **All Belongs to java.util package.**

Examples

Here are some examples of where you might use each of these data structures:

Queue: A queue can be used to implement a buffer for incoming network packets, a job queue for a task scheduler, or a print queue.

Queue<String> queue = new LinkedList<>();

Deque: A deque can be used to implement a stack, a queue, or a double-ended buffer.

Deque<String> deque = new LinkedList<>();

Stack: A stack can be used to implement function calls, backtracking algorithms, or a undo/redo stack.

Stack<String> stack = new Stack<>();

HashSet: A HashSet can be used to implement a cache of unique values, a set of unique identifiers, or a set of unique words in a text document.

LinkedHashSet: A LinkedHashSet can be used to implement a cache of unique values that need to be maintained in a specific order, a set of unique identifiers that need to be maintained in a specific order, or a set of unique words in a text document that need to be maintained in a specific order.

LinkedHashSet<String> linkedHashSet = new LinkedHashSet<>();

TreeSet: A TreeSet can be used to implement a sorted set of unique values, a sorted set of unique identifiers, or a sorted set of unique words in a text document.

TreeSet<String> treeSet = new TreeSet<>();

HashMap: A HashMap can be used to implement a cache of key-value pairs, a map of unique values, or a dictionary.

LinkedHashMap: A LinkedHashMap can be used to implement a cache of key-value pairs that need to be maintained in a specific order, a map of unique values that need to be maintained in a specific order, or a dictionary that needs to be maintained in a specific order.

LinkedHashMap<String, Integer> linkedHashMap = new LinkedHashMap<>();

TreeMap: A TreeMap can be used to implement a sorted map of key-value pairs, a sorted map of unique values, or a sorted dictionary.

TreeMap<String, Integer> treeMap = new TreeMap<>();

Kafka

Kafka Cluster:
- A Kafka cluster consists of Kafka brokers working together to store and distribute data.
- A Kafka broker is a server that stores and distributes streamed data.

Topics and Partitions:
- Topics are logical channels for data streams in Kafka.
- Producers publish messages to topics, and consumers consume messages from topics.
- Partitioning is the process of dividing a topic into multiple partitions.
- Each partition is a separate sequence of messages, allowing distribution across multiple brokers for horizontal scaling.

Offsets:
- Offset is a unique identifier for each message in a topic.
- Consumers use offsets to track processed and pending messages.

Key Elements Relationship:

- A Kafka cluster has one or more Kafka brokers.
- A topic can be partitioned into multiple partitions, each stored on one or more brokers.
- Each message in a partition has a unique offset.
- Consumers use offsets to track processed and pending messages.

Entities:
- Producers: Applications that publish data to Kafka topics.
- Consumers: Applications that consume data from Kafka topics.
- ACLs (Access Control Lists): Used to control access to Kafka topics and other entities.
- Transactions: Ensure atomic processing of groups of messages.

Amazon Elastic Compute Cloud (EC2):
- EC2 provides secure, resizable compute capacity in the cloud.
- Offers a variety of instance types for different compute needs.
- Allows scaling instances up or down based on demand.

Amazon Elastic File System (EFS):
- EFS is a scalable file system service providing elastic NFS file shares.
- Fully managed, scales to petabytes, highly available, and durable.

Scaling Concepts:
- Horizontal Scaling: Adding or removing instances of an application.
- Vertical Scaling: Adjusting resources allocated to an instance by adding or removing CPU, memory, or storage.

AWS Lambda:
- Serverless compute service for running code without managing servers.
- Useful for building event-driven applications.

AWS Parameter Store:
- Secure, hierarchical store for managing application parameters.
- Stores passwords, API keys, and database connection strings.
- Parameters can be organized hierarchically and encrypted at rest and in transit.
- Integrated with other AWS services like AWS Secrets Manager and AWS Systems Manager.

MicroServices Architecture:

1. **Service Development:**
   - Developed independently by small, cross-functional teams.
   - Services perform specific business functions and deploy independently.
2. **Service Registry:**
   - Keeps track of available microservices.
   - Helps services discover and communicate dynamically.
3. **Service Deployment:**
   - Each microservice deployed independently, often in containers.
   - Uses tools like Kubernetes for orchestration.
4. **API Gateway:**
   - Central entry point managing requests.
   - Handles authentication, load balancing, and routing.
5. **Communication:**
   - Microservices communicate via well-defined APIs.
   - Methods include HTTP/REST, message queues, or gRPC.
6. **Data Management:**
   - Each microservice manages its own database.
   - Uses a mix of databases like relational, NoSQL, or in-memory.
7. **Resilience and Fault Tolerance:**
   - Designed to be resilient; failure in one service doesn't affect the entire app.
   - Uses techniques like circuit breakers, retries, and fallback mechanisms.

8. **Scaling:**
    - Microservices scale independently based on needs.
    - Container orchestration tools assist in dynamic scaling.
9. **CI/CD:**
    - CI/CD pipelines for automated testing, building, and deployment.
    - Changes trigger a microservice's deployment pipeline without affecting others.
10. **Monitoring and Logging:**
    - Crucial for understanding health and performance.
    - Tools like Prometheus, Grafana, ELK stack are commonly used.
11. **Security:**
    - Each microservice implements its own security measures.
    - Involves access control, authentication, and data encryption.
12. **Event Sourcing and CQRS:**
    - Some architectures use event sourcing for the primary source of truth.
    - CQRS separates command and query responsibilities.

Summary:

- Microservices are small, independent services.
- They're developed, deployed, and scaled independently.
- Communication via well-defined APIs.
- Resilient to failures with individual scaling.
- CI/CD, monitoring, and security are integral.

Remember the key principles: independence, API communication, resilience, independent scaling, and a strong focus on CI/CD, monitoring, and security.

Spring Security:

1. **Spring Security Overview:**
    - *Definition:* Spring Security is a framework for securing Spring applications.
    - *Example:* Imagine Spring Security as a fortress protecting a city (your application) with its robust walls and guards.
2. **Key Features of Spring Security:**
    - *Authentication Example:* A user providing a username and password to log in.
    - *Authorization Example:* Allowing a user with the "ADMIN" role to access an admin dashboard.
3. **Authentication vs. Authorization:**
    - *Authentication Example:* Showing your ID card at the entrance of a building to prove who you are.
    - *Authorization Example:* Permissions to access different floors within the building based on your role.
4. **Types of Authentication Mechanisms:**
    - *Basic Authentication Example:* Sending a username and password in plain text (like a postcard).
    - *OAuth 2.0 Example:* Logging into a website using your Google account credentials.
5. **Types of Authorization Mechanisms:**
    - *Role-Based Access Control (RBAC) Example:* Assigning "GUEST" and "ADMIN" roles to users.
    - *Expression-Based Access Control (EL) Example:* Allowing access if the user has the "READ" permission.
6. **Security Context:**
    - *Example:* Think of the security context as a badge worn by a person, containing their identity and access permissions.
7. **Security Filter:**

- *Example:* Imagine a security guard at the entrance of a building checking everyone who enters.

8. **Security Filter Chain:**
    - *Example:* Visualize a series of security gates that a person must pass through, each gate performing a different security check.
9. **Basic Authentication vs. Digest Authentication:**
    - *Basic Authentication Example:* Shouting your credentials in a language everyone understands (like shouting your name).
    - *Digest Authentication Example:* Handing over a secret handshake that only a few people understand.

Note:

- Practice to configure Security in Spring boot Authentication logic .

Spring Boot:

Definition:

Spring Boot is a framework for creating stand-alone, production-grade Spring applications. It simplifies development with auto-configuration, starter dependencies, and embedded servers.

Advantages:

- **Reduced Development Time:**
    - *Example:* Spring Boot eliminates boilerplate code, allowing developers to focus on business logic.
- **Increased Productivity:**
    - *Example:* Auto-configuration reduces the need for manual setup, boosting development speed.
- **Improved Code Quality:**
    - *Example:* Spring Boot encourages best practices, enhancing code readability and maintainability.
- **Simplified Configuration and Deployment:**
    - *Example:* Externalized configuration in **application.properties** makes configuration management easier.
- **Enhanced Scalability and Reliability:**
    - *Example:* Built-in features like embedded servers simplify deployment and enhance scalability.

11. Key Features of Spring Boot:

- **Auto-configuration:**
    - *Example:* Spring Boot automatically configures the database connection based on dependencies.
- **Starter Dependencies:**
    - *Example:* Including **spring-boot-starter-web** gets a project started with web development features.
- **Embedded Servers:**
    - *Example:* Tomcat or Jetty can be embedded directly within the application for standalone execution.
- **Actuator:**
    - *Example:* Actuator provides endpoints for monitoring and managing the application.
- **Spring Boot Cloud:**
    - *Example:* Integration with cloud services for easy deployment and scaling.

13. @SpringBootApplication Annotation:

- **Definition:**
    - *Example:* **@SpringBootApplication** combines **@Configuration**, **@EnableAutoConfiguration**, and **@ComponentScan**.

14. Auto-configuration:

- **Definition:**
  - *Example:* Spring Boot automatically configures the datasource based on classpath entries.

15. Starter Dependencies:

- **Definition:**
  - *Example:* Including **spring-boot-starter-data-jpa** sets up JPA-related dependencies.

16. Different Ways to Configure a Spring Boot Application:

- **Using application.properties File:**
  - *Example:* Configuring datasource properties like **spring.datasource.url**.
- **Using @Configuration Class:**
  - *Example:* Creating a class annotated with **@Configuration** to define custom beans.
- **Using @Bean Annotation:**
  - *Example:* Defining a bean with **@Bean** in a configuration class.
- **Using Environment Interface:**
  - *Example:* Retrieving properties using **@Value** annotation and **Environment** interface.

17. Creating a Simple Spring Boot Web Application:

- **Steps:**
  1. *Example:* Create a new Spring Boot project using Spring Tool Suite.
  2. *Example:* Add **spring-boot-starter-web** dependency.
  3. *Example:* Create a **@RestController** class with a mapped URL method.
  4. *Example:* Run the Spring Boot application.

18. Handling HTTP Requests in Spring Boot:

- **Ways:**
  - *@RequestMapping, @GetMapping, @PostMapping, @PutMapping, @DeleteMapping.*

19. RestController vs Controller:

Annotation	Purpose	Example
@RestController	Used to create RESTful web services, returns data directly	Returning JSON directly from methods
@Controller	Used for handling web requests, generates views or redirects	Generating views or redirecting to other URLs

20. Using Spring Boot to Implement RESTful APIs:

- **Steps:**
  - *Example:* Use **@RestController** to handle HTTP requests and return responses in JSON.

21. Configuring Spring Boot for a Different Web Server:

- **Steps:**
  - *Example:* Set **server.port** property to configure a different web server.

22. Configuring a Database Connection in Spring Boot:

- **Steps:**
  - *Example:* Set **spring.datasource.url** property in **application.properties**.

What Does @SpringBootApplication Annotation Do Internally:

- **Functionality:**

- *Example:* Internally combines **@Configuration**, **@EnableAutoConfiguration**, and **@ComponentScan**.

Effects of Running Spring Boot Application as "Java Application":

- **Outcome:**
  - *Example:* Automatically launches the embedded Tomcat server.

Spring Boot Dependency Management System:

- **Role:**
  - *Example:* Manages dependencies and configuration automatically without specifying versions.

Validation Approaches for Web Service Endpoints:

1. **Use Validation Annotations:**
   - *Spring Validation Annotations:*
     - Employ annotations like **@NotNull**, **@Size**, **@Email** on method parameters or DTO classes.
     - Example:

```
@PostMapping("/createUser")
public ResponseEntity<String> createUser(@RequestBody @Valid UserDto userDto) {
    // Business logic
}
```

   - *Hibernate Validator:*
     - If using Spring Boot with Hibernate Validator, use **@Valid** to trigger validation.
     - Example:

```
public class UserDto {
    @NotNull
    @Size(min = 3, max = 50)
    private String username;
    // other fields and getters/setters
}
```

2. **Custom Validation Logic:**
   - *Implement Custom Validator:*
     - Create a class implementing the **Validator** interface for custom validation.
     - Example:

```
public class UserValidator implements Validator {
    // Implementation
}
```

3. **Exception Handling:**
   - *Handle Validation Errors:*
     - Customize exception handling to return appropriate error responses.
     - Example:

```
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<String> handleValidationException(MethodArgumentNotValidException ex) {
    // Handle and respond to validation errors
}
```

4. **Integration Testing:**
   - *Write Integration Tests:*
     - Develop tests with valid and invalid input data to verify expected outcomes.

5. **Swagger/OpenAPI Documentation:**
    - *Use Documentation Tools:*
        - Utilize Swagger or OpenAPI to document the API, specifying validation constraints.
        - Acts as a reference for API consumers.
6. **Security Considerations:**
    - *Sanitize Inputs:*
        - Protect against malicious input by sanitizing inputs.
        - Implement robust security measures.

@Qualifier and Related Concepts:

- **@Qualifier Annotation:**
    - Used in Spring to specify which bean should be autowired when multiple implementations are available.
    - Applied on the injection point, along with the **@Autowired** annotation.
- **Related Concepts:**

1. **@Primary Annotation:** Marks a bean as primary when multiple candidates exist.
2. **@Resource Annotation:** Used for autowiring by name.
3. **@Autowired Annotation:** Used for automatic dependency injection.

Note: These concepts are widely used in the Spring framework for managing dependencies and beans.

RestAPI

23. RESTful Web Services:

- **Definition:**
    - *Example:* RESTful web services use HTTP to access resources, following the principles of Representational State Transfer.
- **Features:**
    - *Example:* URIs, HTTP methods, messaging, and resources play crucial roles in RESTful architecture.

24. Differences between @Controller and @RestController:

- **@Controller:**
    - *Example:* Used for handling web requests and often returns views or redirects.
- **@RestController:**
    - *Example:* A combination of @Controller and @ResponseBody, specifically used for returning data.

25. HTTP Methods:

- **GET:**
    - *Example:* Fetching user details from the server.
- **POST:**
    - *Example:* Creating a new user on the server.
- **PUT:**
    - *Example:* Updating an existing user on the server.
- **DELETE:**
    - *Example:* Deleting a user from the server.
- **PATCH:**
    - *Example:* Modifying specific details of a user on the server.

26. Differences between PUT and POST in REST:

- **PUT:**
  - *Example:* Used for updating or replacing existing resources.
- **POST:**
  - *Example:* Used for creating new resources on the server.

27. @PathVariable Annotation:

- **Definition:**
  - *Example:* Used to extract values from the URI and pass them to the controller method.

28. Circular Dependency:

- **Definition:**
  - *Example:* Bean A depends on Bean B, and Bean B depends on Bean A, forming a circular relationship.

29. Logging in Java:

- **Definition:**
  - *Example:* Java Logging API provides a systematic way to record application events for troubleshooting.
- **Need for Logging:**
  - *Example:* Logging helps trace and identify errors, critical failures, and provides application insights.

30. Project Object Model (POM):

- **Definition:**
  - *Example:* The **pom.xml** file contains project information and configuration for Maven build.

31. Spring Beans:

- **Definition:**
  - *Example:* Spring beans are Java objects managed by the IoC container, instantiated, configured, and wired.

32. Bean Scopes in Spring:

- **Singleton:**
  - *Example:* Single instance per IoC container.
- **Prototype:**
  - *Example:* Any number of object instances per bean definition.
- **Request:**
  - *Example:* Scope of bean definition is an HTTP request.
- **Session:**
  - *Example:* Scope of bean definition is an HTTP session.
- **Global-session:**
  - *Example:* Scope of bean definition is a global HTTP session.

33. Spring AOP (Aspect-Oriented Programming):

- **Definition:**
  - *Example:* AOP provides modularity by separating cross-cutting concerns into aspects.

1. PUT (Update/Replace Resource):

- **Purpose:**

- Used to update or replace an existing resource or create a new resource if it doesn't exist.

- **Request Body:**
  - Typically contains the full representation of the resource.
- **Idempotent:**
  - Generally considered idempotent. Repeated identical requests have the same effect as a single request.
- **Example:**

```
httpCopy code
PUT /users/123 Content-Type: application/json
{ "id": 123, "name": "John Doe", "age": 30 }
```

2. PATCH (Partial Update of Resource):

- **Purpose:**
  - Used to apply partial modifications to an existing resource.
- **Request Body:**
  - Contains the set of changes to be applied to the resource, rather than the full representation.
- **Idempotent:**
  - Not guaranteed to be idempotent. Repeated identical requests may have different effects.
- **Example:**

```
httpCopy code
PATCH /users/123 Content-Type: application/json
{ "age": 31 }
```

3. POST (Create Resource):

- **Purpose:**
  - Used to submit data to be processed to a specified resource, resulting in the creation of a new resource.
- **Request Body:**
  - Contains the data to be added to the resource.
- **Idempotent:**
  - Generally not idempotent. Repeated identical requests may result in different resources being created.
- **Example:**

```
httpCopy code
POST /users Content-Type: application/json
{ "name": "Jane Doe", "age": 25 }
```

Summary:

- **PUT:** Used for full updates or replacements of existing resources. Idempotent.
- **PATCH:** Used for partial updates of existing resources. Not guaranteed to be idempotent.
- **POST:** Used for creating new resources. Generally not idempotent.

Annotation:

Spring MVC Annotations:

1. **@GetMapping:**
   - Maps HTTP GET requests to a specific handler method.
   - Creates a web service endpoint for fetching data.
   - *Example:* Retrieving user details.
2. **@PostMapping:**
   - Maps HTTP POST requests to a specific handler method.
   - Creates a web service endpoint for creating resources.

- *Example:* Creating a new user.

3. **@PutMapping:**
   - Maps HTTP PUT requests to a specific handler method.
   - Creates a web service endpoint for creating or updating resources.
   - *Example:* Creating or updating user details.
4. **@DeleteMapping:**
   - Maps HTTP DELETE requests to a specific handler method.
   - Creates a web service endpoint for deleting a resource.
   - *Example:* Deleting a user.
5. **@PatchMapping:**
   - Maps HTTP PATCH requests to a specific handler method.
   - *Example:* Handling partial updates.
6. **@Autowired:**
   - Enables annotation-based auto-wiring in Spring.
   - Used to autowire Spring beans on setter methods, instance variables, and constructors.
   - *Example:* Automatically injecting dependencies.
7. **@Configuration:**
   - Class-level annotation used by Spring Containers as a source of bean definitions.
   - *Example:* Configuring application beans.
8. **@ComponentScan:**
   - Used with **@Configuration** to scan a package for beans.
   - *Example:* Scanning for components in a specific package.
9. **@Bean:**
   - Method-level annotation indicating a method produces a bean to be managed by the Spring Container.
   - *Example:* Defining a bean for dependency injection.
10. **@Controller:**
    - Class-level annotation marking a class as a web request handler.
    - Typically used to serve web pages.
    - *Example:* Handling web requests and returning views.
11. **@RestController:**
    - Combination of **@Controller** and **@ResponseBody** annotations.
    - Eliminates the need for annotating each method with **@ResponseBody**.
    - *Example:* Handling web requests and directly returning data.
12. **@RequestMapping:**
    - Used to map web requests.
    - Optional elements like **consumes**, **header**, **method**, **name**, **params**, **path**, **produces**, and **value**.
    - *Example:* Mapping URLs at class and method levels.
13. **@RequestParam:**
    - Extracts query parameters from the URL.
    - Suitable for web applications.
    - *Example:* Retrieving query parameters with default values.
14. **@Service:**
    - Class-level annotation indicating a class contains business logic.
    - *Example:* Defining a service class.
15. **@Repository:**
    - Class-level annotation for Data Access Objects (DAOs) that access the database directly.
    - *Example:* Handling database operations.
16. **@PathVariable:**
    - Used to extract values from the URI.
    - Suitable for RESTful web services with path variables.
    - *Example:* Extracting values from the URI.
17. **@ExceptionHandler:**

- Handles exceptions in Spring Boot applications.
- Can handle specific exceptions or all exceptions.
- *Example:* Handling specific and all exceptions.

Additional Tips:

- **On @ExceptionHandler:**
  - Can be used in any class but typically used in controllers.
  - Multiple annotations can be used on a single method to handle different exceptions.

SQL:

1. **Basic Retrieval:**
   - Retrieve all columns from a specific table.

```
SELECT * FROM your_table;
```

2. **Filtering and Sorting:**
   - Retrieve employees with a salary greater than a certain value, sorted by salary in descending order.

```
SELECT * FROM employees WHERE salary > 50000 ORDER BY salary DESC;
```

3. **Join Operations:**
   - Retrieve customer names and their orders using an INNER JOIN.

```
SELECT customers.name, orders.order_id FROM customers INNER JOIN orders ON customers.customer_id = orders.customer_id;
```

4. **Grouping and Aggregation:**
   - Calculate the average salary for each department.

```
SELECT department, AVG(salary) AS avg_salary FROM employees GROUP BY department;
```

5. **Subqueries:**
   - Retrieve employees who have a salary greater than the average salary.

```
SELECT * FROM employees WHERE salary > (SELECT AVG(salary) FROM employees);
```

6. **Updating Records:**
   - Update the salary of an employee with a specific ID.

```
UPDATE employees SET salary = 60000 WHERE employee_id = 123;
```

7. **Inserting Records:**
   - Insert a new employee record.

```
INSERT INTO employees (employee_id, name, salary, department) VALUES (124, 'John Doe', 55000, 'IT');
```

8. **Deleting Records:**
   - Delete records of employees who have left the company.

```
DELETE FROM employees WHERE employment_status = 'Left';
```

9. **Views:**
   - Create a view that displays the employee ID, name, and department for reporting purposes.

```
CREATE VIEW employee_report AS SELECT employee_id, name, department FROM employees;
```

10. **Constraints and Indexing:**
    - Create a table with primary key and foreign key constraints.

CREATE TABLE orders (

```
  order_id INT PRIMARY KEY,
  customer_id INT,
  order_date DATE,
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);
```

11. **Using LIKE for Pattern Matching:**
    - Retrieve employees whose names start with "J".

```
SELECT * FROM employees WHERE name LIKE 'J%';
```

12. **Counting Rows:**
    - Count the number of employees in each department.

```
SELECT department, COUNT(*) AS employee_count FROM employees GROUP BY department;
```

13. **Top N Records:**
    - Retrieve the top 5 highest-paid employees.

```
SELECT * FROM employees ORDER BY salary DESC LIMIT 5;
```

14. **Date Functions:**
    - Retrieve orders placed in the last month.

```
SELECT * FROM orders WHERE order_date >= CURRENT_DATE - INTERVAL 1 MONTH;
```

15. **Using CASE Statements:**
    - Classify employees into salary ranges.

```
SELECT name, salary,
    CASE
      WHEN salary < 50000 THEN 'Low'
      WHEN salary >= 50000 AND salary < 80000 THEN 'Medium'
      ELSE 'High'
    END AS salary_range
FROM employees;
```

16. **Self-Join:**
    - Retrieve pairs of employees who have the same manager.

```
SELECT e1.name AS employee1, e2.name AS employee2
FROM employees e1
INNER JOIN employees e2 ON e1.manager_id = e2.manager_id AND e1.employee_id <
e2.employee_id;
```

17. **Using IN Clause:**
    - Retrieve orders for specific customers.

```
SELECT * FROM orders WHERE customer_id IN (101, 102, 105);
```

18. **Window Functions (e.g., ROW_NUMBER()):**
    - Assign a unique rank to each employee based on their salary within each department.

```
SELECT employee_id, name, salary, department,
    ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS salary_rank
FROM employees;
```

19. **Handling NULL Values:**
    - Retrieve employees with a specified department or those without a department.

```
SELECT * FROM employees WHERE department = 'IT' OR department IS NULL;
```

20. **Using EXISTS:**
    - Retrieve customers who have placed at least one order.

```
SELECT * FROM customers WHERE EXISTS (SELECT 1 FROM orders WHERE
orders.customer_id = customers.customer_id);
```